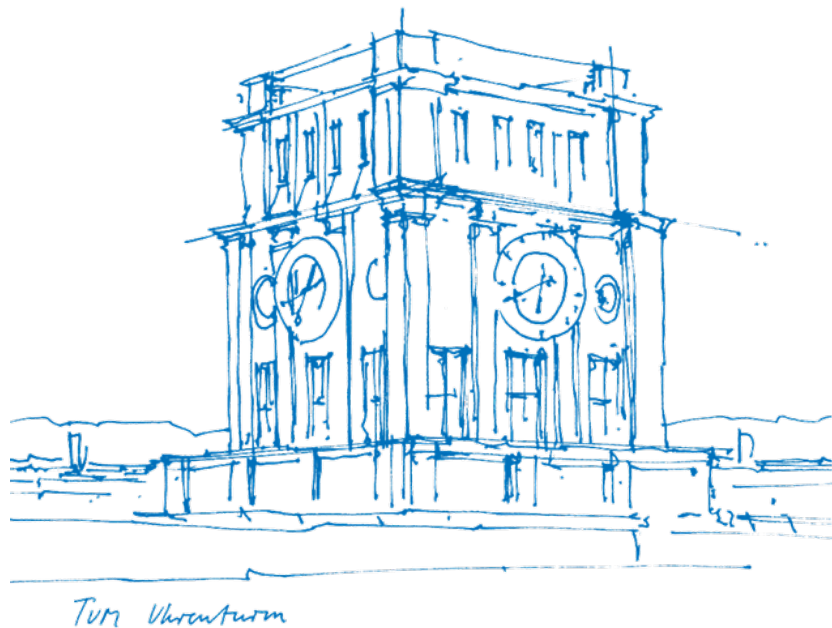


## Master's Thesis in Information Systems

Xin Wen

# Document-Based Process Modeler





## Master's Thesis in Information Systems

Xin Wen

# Document-Based Process Modeler

Dokumentatenbasierte Bearbeitung und Änderung von  
Prozessmodellen

Thesis for the Attainment of the Degree  
**Master of Science**

at the TUM School of Computation, Information and Technology,  
Department of Computer Science,  
Chair of Information Systems and Business Process Management (i17)

### Examiner

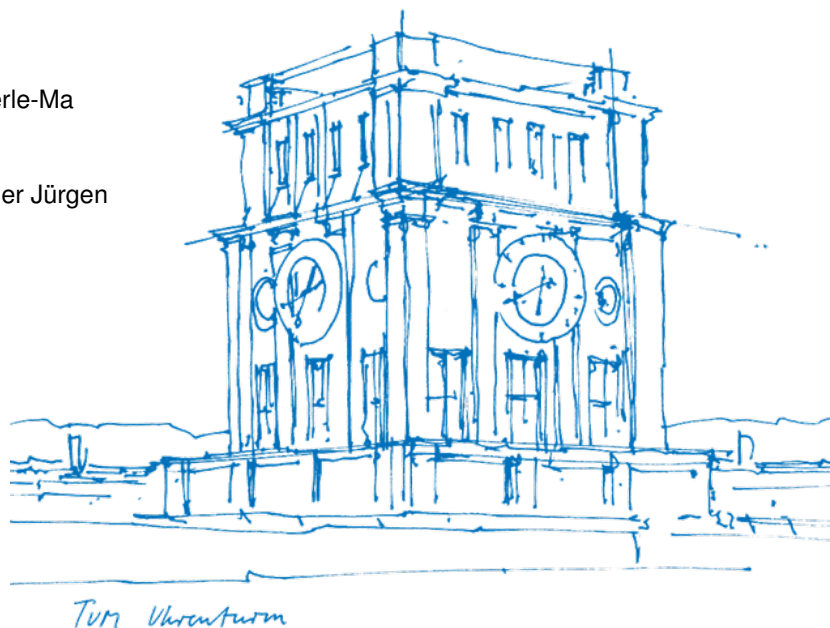
Prof. Dr. Stefanie Rinderle-Ma

### Supervised by

Dr. rer. soc. oec. Mangler Jürgen

### Submitted on

15.04.2026



# Declaration of Academic Integrity

I confirm that this master's thesis is my own work and I have documented all sources and material used.

I am aware that the thesis in digital form can be examined for the use of unauthorized aid and in order to determine whether the thesis as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future theses submitted. Further rights of reproduction and usage, however, are not granted here.

This thesis was not previously presented to another examination board and has not been published.

Garching, 15.04.2026

Xin Wen

## Abstract

Business processes are valuable assets of organizations, yet most process knowledge is stored in unstructured formats such as Word documents and PDF files. Manual transformation from unstructured documents into structured process models is time-consuming and error-prone. Large language models (LLMs) have proven effective in supporting efficient process modeling, but they cannot fully replace human experts, especially in domain-specific scenarios. Hybrid human-LLM process modeling is therefore a promising approach to leverage the strengths of both sides and compensate for their respective weaknesses. However, existing tools and approaches still focus on model generation capabilities and quality, making it still inconvenient for the use scenario of creating process models from documents. This work addresses these limitations by proposing and developing an integrated web-based system that supports document-driven process modeling with LLMs through a more seamless, efficient, and user-friendly workflow. The system unifies document management, direct text selection, automated model generation, visual traceability between text and models, subprocess relationship establishment, and model version control within a single workspace. A user evaluation with four process modeling experts shows that the system is intuitive, usable, and valuable for multi-model and document-intensive scenarios.

**Keywords:** *Business Process Management, Process Modeling*

# Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                              | <b>7</b>  |
| 1.1      | Motivation . . . . .                             | 7         |
| 1.2      | Research Questions . . . . .                     | 7         |
| 1.3      | Contribution . . . . .                           | 8         |
| 1.4      | Methodology . . . . .                            | 8         |
| 1.5      | Evaluation . . . . .                             | 9         |
| 1.6      | Structure . . . . .                              | 9         |
| <b>2</b> | <b>Related Work</b>                              | <b>9</b>  |
| 2.1      | Foundational Concepts and Technologies . . . . . | 9         |
| 2.2      | AI-Assisted Process Modeling Tools . . . . .     | 9         |
| 2.3      | AutoBPMN.AI . . . . .                            | 12        |
| <b>3</b> | <b>Solution Design</b>                           | <b>13</b> |
| 3.1      | Requirements . . . . .                           | 13        |
|          | Context and Goal . . . . .                       | 13        |
|          | Requirements Elicitation . . . . .               | 13        |
| 3.2      | User Interface and Interaction Design . . . . .  | 16        |
| 3.3      | Modeling Workflow Design . . . . .               | 16        |
|          | Design Difficulties . . . . .                    | 17        |
| 3.4      | System Architecture . . . . .                    | 17        |
| <b>4</b> | <b>Implementation</b>                            | <b>18</b> |
| 4.1      | Frontend . . . . .                               | 18        |
|          | Architecture . . . . .                           | 18        |
|          | State Management . . . . .                       | 20        |
|          | Document Processing . . . . .                    | 21        |
|          | Text Selection Rendering . . . . .               | 22        |
|          | Modeling . . . . .                               | 24        |
|          | Rendering Customization . . . . .                | 25        |

|                                                          |           |
|----------------------------------------------------------|-----------|
|                                                          | 5         |
| 4.2 Backend . . . . .                                    | 25        |
| Architecture . . . . .                                   | 25        |
| 4.3 Database Schema . . . . .                            | 28        |
| <b>5 Evaluation</b>                                      | <b>29</b> |
| 5.1 Evaluation Procedure . . . . .                       | 29        |
| 5.2 Evaluation Results . . . . .                         | 30        |
| Overall Perception and Task Completion . . . . .         | 30        |
| Positive Aspects . . . . .                               | 31        |
| Problems in Interaction and Use . . . . .                | 32        |
| Potential Improvements and Additional Features . . . . . | 34        |
| Future Use Intention . . . . .                           | 34        |
| <b>6 Discussion</b>                                      | <b>35</b> |
| 6.1 Usability and User Experience . . . . .              | 35        |
| 6.2 User Expectations . . . . .                          | 35        |
| 6.3 Limitations . . . . .                                | 36        |
| <b>7 Conclusion</b>                                      | <b>36</b> |
| <b>Bibliography</b>                                      | <b>38</b> |
| <b>A Appendix</b>                                        | <b>40</b> |
| Design Constraints . . . . .                             | 40        |

## List of Tables

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| 1 Literature review: horizontal research . . . . .                          | 10 |
| 2 Related work on AI-assisted process modeling tools (simplified) . . . . . | 12 |
| 3 Summary of Evaluation Results by Functionality . . . . .                  | 33 |

## List of Figures

|                            |    |
|----------------------------|----|
| 1 System context . . . . . | 14 |
| 2 Usage model . . . . .    | 15 |

|    |                                                                   |    |
|----|-------------------------------------------------------------------|----|
| 3  | Conceptual data model . . . . .                                   | 15 |
| 4  | Workspace UI Layout . . . . .                                     | 16 |
| 5  | System architecture . . . . .                                     | 17 |
| 6  | Frontend codebase structure . . . . .                             | 18 |
| 7  | Simplified workspace initialization flow . . . . .                | 20 |
| 8  | store implementation . . . . .                                    | 21 |
| 9  | Core implementation of selection serialization . . . . .          | 23 |
| 10 | Example JSON schema used in the backend for text quotes . . . . . | 26 |
| 11 | Backend Component . . . . .                                       | 28 |
| 12 | Modeling result . . . . .                                         | 30 |
| 13 | Modeling result . . . . .                                         | 31 |
| 14 | Different modeling strategies . . . . .                           | 32 |



# Introduction

## Motivation

Business processes represent core operational assets of modern organizations, defining standardized workflows, responsibilities, and execution logic for daily operations. However, a large proportion of enterprise process knowledge remains trapped in unstructured document formats, such as Word, PDF, plain text, and internal manuals. Manually converting such unstructured textual content into formal, executable process models is labor-intensive, time-consuming, and prone to human error, reducing the efficiency of process design and digitization.

In the era of generative artificial intelligence (AI), large language models (LLMs) have emerged as powerful enablers for automated process modeling, demonstrating strong capabilities in understanding natural language, extracting logical structures, and generating standardized process notations. Nevertheless, LLMs cannot fully replace human experts, especially in domain-specific scenarios requiring professional knowledge, contextual judgment, and structural decision-making[1]. Therefore, hybrid human-LLM process modeling has become a promising paradigm that combines human domain expertise with AI efficiency.

Despite rapid advancements, existing LLM-assisted process modeling solutions suffer from critical limitations in document-driven scenarios. Most tools adopt a “single-view, single-model” paradigm, which lacks scalability for complex systems involving multiple processes and subprocesses. To overcome these gaps, this thesis proposes an integrated tool, Document-Based Process Modeler. The tool enables a seamless, efficient, and user-centric modeling workflow with three core innovations: unified workspace management for correlated documents and models; direct text selection from uploaded documents as LLM input; and side-by-side visualization to establish explicit visual links between source text and generated process models to make a more intuitive and verifiable modeling experience.

## Research Questions

This thesis addresses the following research questions:

RQ1 How do existing tools support AI-assisted process modeling multiple documents as input?

RQ2: What functionalities should a system provide to support AI-assisted process modeling from multiple documents, and how should these functionalities be designed and integrated in a seamless and user-friendly manner?

RQ3 How do user perceive the benefits and limitation of the system?

### **Contribution**

The contribution of this thesis consists of the following aspects:

Firstly, a systematic review of the existing tools and approaches for AI-assisted process modeling is conducted, which provides insights into the current state of the art and guides the design of the proposed system.

Secondly, an integrated system is designed and developed to support process modeling from documents using LLMs. The system offers a more seamless, efficient, and user-friendly workflow by integrating multiple functionalities, such as document management, model generation, and traceability between text and model.

Thirdly, the proposed system is evaluated through a user study with process modeling experts. The evaluation results provide insights into the system's benefits and limitations from the users' perspective, which can inform future improvements and further research in this area.

### **Methodology**

The process of this thesis employs the research cycles of DSR from Hevner[2]: The relevance cycle: the first cycle is done through the communication with modeling experts and proposes the tool and elicits the requirements in section 3.1 for the system.

The rigor cycle: it is done through the review in section 2. The design cycle: the first cycle is done through the delivered solution of RQ2 and evaluation result of RQ3. Through the analysis and discussion of the evaluation result, we can identify the strengths and weaknesses of the proposed tool, and further improve the design of the tool in the next iteration.

## **Evaluation**

The evaluation of the proposed system is done with 4 experts in qualitative way. It consisted of task-based activities and a follow-up questionnaire to collect user feedback on the tool's usability and perceived usefulness.

## **Structure**

The remainder of this thesis is structured as follows:

Chapter 2 reviews the related work of LLM-based process modeling tools. Chapter 3 presents the design solution of the tool, including its requirements, architecture, and functionalities. Chapter 4 describes its technical implementation. Chapter 5 describes the evaluation of the proposed system, including the methodology, results. Chapter 6 discusses the implications of the findings and potential future work. Finally, Chapter 7 summarizes the current work and outlines future work.

# **Related Work**

## **Foundational Concepts and Technologies**

In this section, we briefly introduce the foundational concepts and technologies relevant to this thesis.

BPM: BPMN: RPST: AI: Natural Language Processing: Generative AI: LLM:

## **AI-Assisted Process Modeling Tools**

While many tools exist that deal with analysis of large document sets, to the best of our knowledge no integrated tool with the dedicated purpose of creating Process Models from documents set exists. Therefore here we will explore some related tools.

In total, eleven AI-assist modeling tools are explored during the literature review (See table 2). Here, we mainly focus on the key approaches and the distinguishing features they offer.

In the 12 hits of the first search query("allintitle: process modeling generative AI"), the tool ProMoAI [3] is identified. It is the first tool shown in the academic literature to leverage LLMs for automated

**Table 1***Literature review: horizontal research*

| <i>Keywords for search</i>                  | <i>hits</i> | <i>selected</i> | <i>Selection criteria</i> |
|---------------------------------------------|-------------|-----------------|---------------------------|
| allintitle: process modeling generative AI  | 12          | 1               | Process modeling tool     |
| allintitle: process modeling LLM[s]         | 16+7        | 5+0             | Process modeling tool     |
| allintitle: process modeling large language | 49          | 1               |                           |
| allintitle: conversational process modeling | 8           | 1               | Process modeling tool     |
| allintitle: BPMN generative AI              | 2           | 0               |                           |
| allintitle: BPMN LLM[s]                     | 12+8        | 2+1             |                           |
| allintitle: BPMN large language             | 7           | 1               |                           |
| <i>Results horizontal search</i>            | 121         | 11              |                           |
| <i>Results vertical/backward search</i>     |             |                 |                           |
| Added papers                                |             | 1               |                           |
| <i>Overall:</i>                             |             | 12              |                           |

Source: <https://scholar.google.com/>

process model generation, introduced in 2024, and is frequently referenced in subsequent work. ProMoAI enables the transformation of natural language into process models and supports iterative refinement through feedback loops, demonstrating the feasibility of LLM-based process modeling.

In the second query ("allintitle: process modeling LLM"), among 16 hits, 5 tools are identified. The first two noticed is BPMN-Chatbot [4] and BPMN Assist [5]. BPMN-Chatbot is an early one in 2024 with key strength in token efficiency [6], compared with ProMoAI's approach. Its method was introduced in. BPMN Assist [5] is a very latest one. BPMN Assistant [5] represents a more recent development, emphasizing structured intermediate representations (e.g., JSON) to improve reliability, editing capabilities, and performance in interactive modeling scenarios.

The tool KICoPro is introduced in [7], focusing on human-in-the-loop process modeling, where users collaborate with AI systems during model creation.

The tool HyperMode is introduced in [8], which combines LLM-based generation with a direct manipulation interface, allowing users to refine process models interactively by selecting and modifying elements within the diagram.

Additionally, the tool BPMNGen is firstly introduced in [9] then further developed and evaluated in [10]. It is a different one using NLP, with empirical evaluations showing strong semantic quality for simple to moderately complex processes.

In the last search attempt for term "process modeling" by combining it with "large language", one more tool PRODIGY is explored mentioned within the paper [11]

In the search query ("allintitle: conversational process modeling"), the only tool AutoBPMN.AI [12] is found.

Finally, combining "BPMN" with "generative AI", "LLM[s]", or "large language" led to the identification of 3 additional tools, including BPMN-Chatbot++ [13], LLM4BPMNGen [14], and Nala2BPMN [15].

BPMN-Chatbot++[13] is found in Tool no name [16]

LLM4BPMNGen[14]

Nala2BPMN [15]

In summary, although a growing number of tools leverage LLMs to support process modeling, none provide a fully integrated solution for automatically deriving complete process models from large document collections.

In summary, while numerous tools leverage LLMs for generating process models from natural language, they predominantly focus on single textual inputs and interactive refinement. There is still no integrated solution specifically designed to process and synthesize large document sets into coherent, unified process models. This highlights a clear research gap and motivates the need for approaches that bridge document analysis and process model generation in a scalable and integrated manner.

**Table 2**  
*Related work on AI-assisted process modeling tools (simplified)*

| Tool           | Year | Input      | Output      | Key Contribution                                      |
|----------------|------|------------|-------------|-------------------------------------------------------|
| ProMoAI        | 2024 | NL text    | BPMN        | Code-based generation with correctness focus          |
| BPMN-Chatbot   | 2024 | Text/voice | BPMN        | Conversational modeling with high token efficiency    |
| Nala2BPMN      | 2024 | NL text    | BPMN        | Hybrid LLM + algorithm pipeline                       |
| PRODIGY        | 2024 | NL + docs  | BPMN (text) | RAG-based enterprise modeling                         |
| LLM4BPMNGen    | 2025 | Multimodal | BPMN        | Multimodal input + structured prompting               |
| BPMN-Chatbot++ | 2025 | Text/voice | BPMN        | Improved prompting and interaction                    |
| AutoBPMN.AI    | 2025 | NL text    | CPEE        | Executable model generation                           |
| HyperMode      | 2025 | NL + UI    | BPMN        | Direct manipulation + LLM interaction                 |
| BPMNGen        | 2026 | NL text    | BPMN        | Conversational generation + human-centered evaluation |
| BPMN Assist    | 2026 | NL text    | BPMN        | JSON-based intermediate for efficient editing         |
| KICoPro        | 2026 | NL + human | BPMN        | Human-in-the-loop modeling                            |

### AutoBPMN.AI

Here, we provide a more detailed in depth review of the AutoBPMN.AI tool, as we will leverage its LLM service for the automated process model generation in our proposed system.

AutoBPMN.AI provides an LLM-backed service that turns natural language process descriptions into an *executable* process representation (CPEE-Tree) [12]. The core interaction is conversational: the user describes a procedure in free-form text, and the service produces (and maintains) a structured model corresponding to that description.

Model creation is performed iteratively over multiple turns. After the initial model draft is generated, the user can issue follow-up instructions (e.g., clarifications, corrections, additions, or restructuring requests). The service uses the conversational context to interpret each instruction and updates the executable model accordingly, enabling step-by-step refinement rather than a single one-shot generation.

In summary, the AutoBPMN.AI LLM service operates as a dialogue-driven modeling loop with two key artifacts exchanged between user and system: natural language messages as input and an updated CPEE-Tree model as output at each iteration.

In this thesis, we will leverage the LLM REST service provided by AutoBPMN.AI for model generation and refinement within our proposed system.

## Solution Design

### Requirements

#### *Context and Goal*

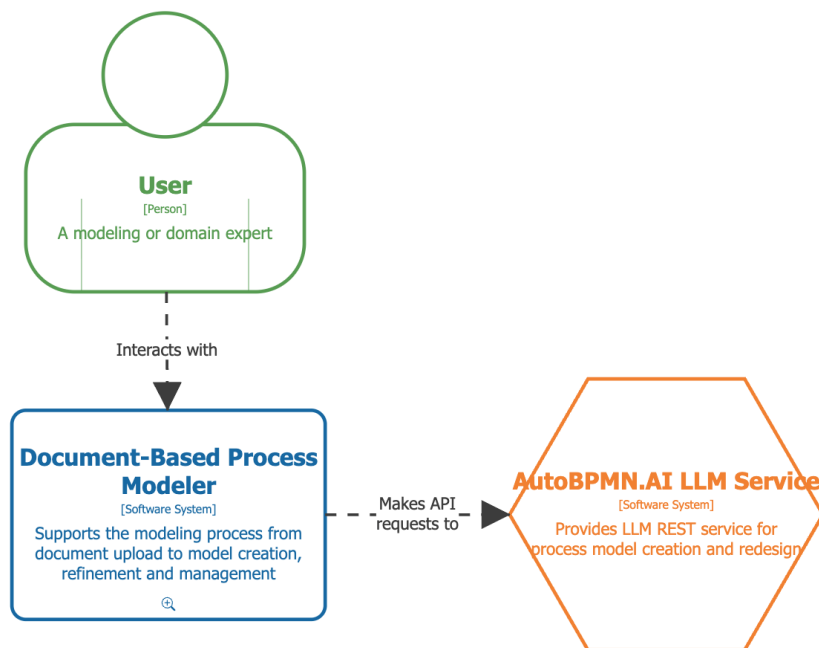
First, the context and objectives of the solution design are defined. The system is provided as a web-based application, as illustrated in Figure 1. The target users include both process modeling experts and domain experts who need to create process models from textual documents. To improve modeling efficiency and support non-expert users, the system leverages the LLM service of AutoBPMN.AI for automated model generation. The overall objective is to provide an integrated platform in which users can complete the entire modeling workflow—from document upload to model creation, modification, and subsequent management—in a seamless and user-friendly manner, thereby addressing RQ2.

#### *Requirements Elicitation*

Functional requirements define the system’s expected behavior by describing the functions, inputs, outputs, and interactions that enable the system to fulfill its intended objectives [17]. In this work, we adopt the classification proposed by [18], further diving the functional requirements into minimal requirements, which represent the indispensable core functionalities required for the system to operate, and enhancing requirements, which include additional features that improve usability and overall performance but are not strictly necessary for basic operation.

When starting the work, the following minimal requirements for the system have been elicited by my supervisor. Firstly, the system should allow users to upload multiple documents. The basic process modelling is defined as follows: first, the user should be able to select multiple passages from a single document; then, by clicking a Generate button, the system will send the selected text

**Figure 1**  
System context



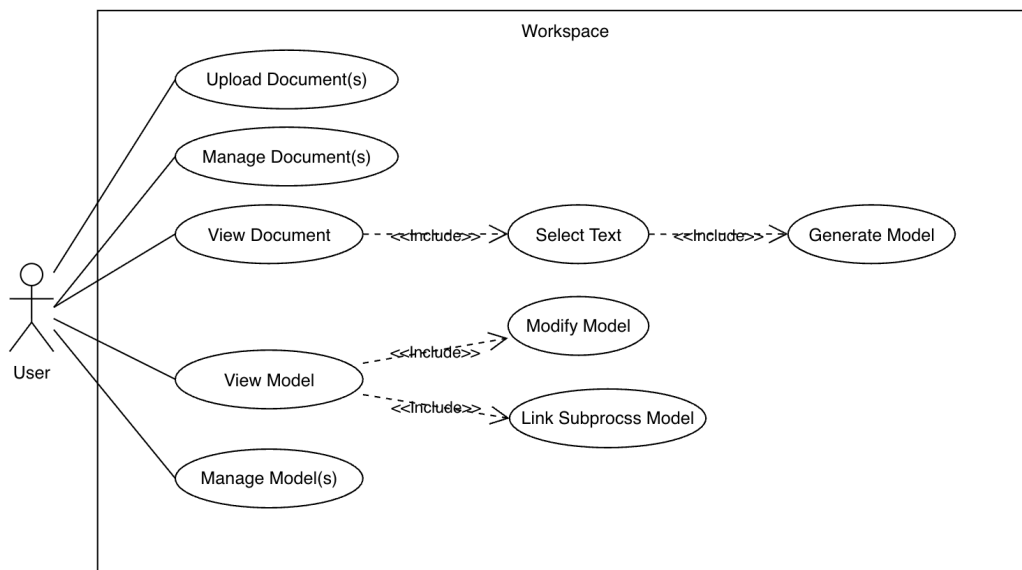
to the LLM service. The system should then display the generated model side-by-side with the document to facilitate verification. Besides, the system should always provide a clear visual link between the text and the generated model. Then a user should be able to select other passages to create more models. Another important requirement is that the system should allow users to establish process-subprocess relationships between models. This flow should be designed on a single web page to avoid users having to switch between tools. The activities described are depicted in the Figure 2. Here, we use the concept of workspace for this dedicated interface, a term commonly used across many software tools, where the modelling flow is performed.

Regarding non-functional requirements, only usability, as a very general goal, is defined at this stage. Besides this, a series of technical design constraints are specified. In addition to the two constraints already introduced in the system context, i.e. the system should be implemented as a web-based application and should leverage AutoBPMN.AI's LLM service, two more constraints are explicitly defined. First, the frontend should be developed using native web technologies (JavaScript, HTML, and CSS). Second, the visualization of the process model should be implemented using the CPEE Layout<sup>1</sup> library, following the same layout as used in AutoBPMN.AI, which makes the models easy to export and import, and also provides the potential for them to be uploaded to the CPEE

<sup>1</sup><https://github.com/etm/cpee-layout>

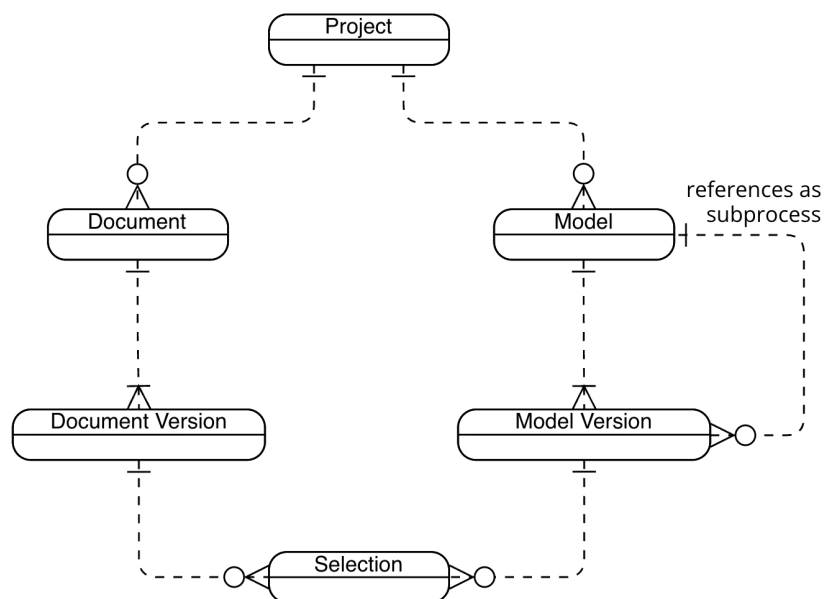


**Figure 2**  
*Usage model*



environment without additional manual effort, where they can be directly further manipulated and executed.

**Figure 3**  
*Conceptual data model*



## User Interface and Interaction Design

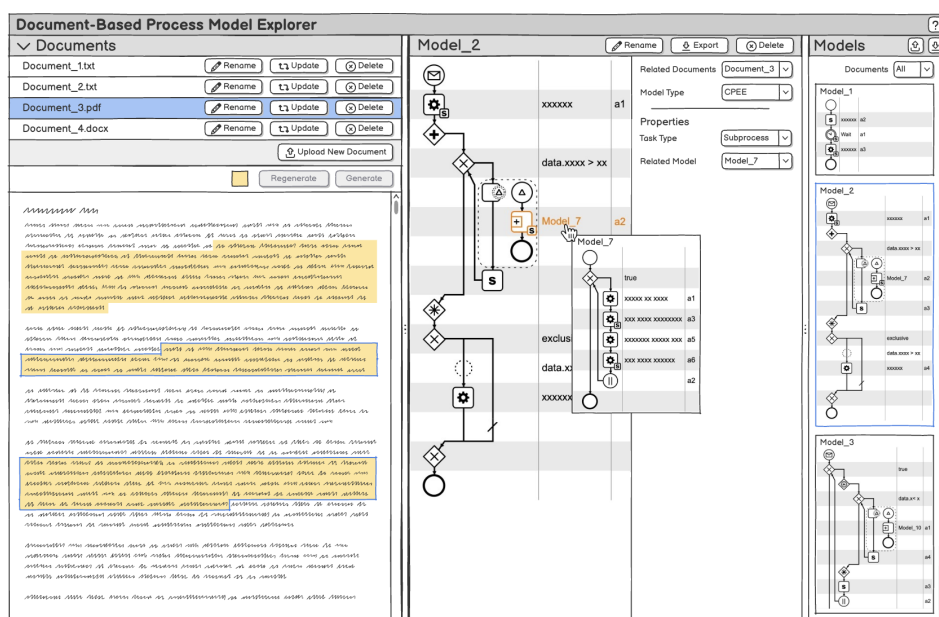
For the user interface and interaction design, the focus is on providing a seamless and intuitive experience for users to interact with the system. The design aims to facilitate efficient document management, process modeling, and visualization of process models.

The design overview is described from the user perspective.

The layout design of the main workspace is shown in the figure 4 in a 3-pane layout. The left pane is for document uploading and viewing, the middle pane is for process modeling and the right pane is for process models viewing and project graph visualization. The details are divided as the following:

a) Document List b) Document Viewer c) Model Editor d) Model List e) Project Graph.

**Figure 4**  
*Workspace UI Layout*



The user interface layout of the workspace page.

## Modeling Workflow Design

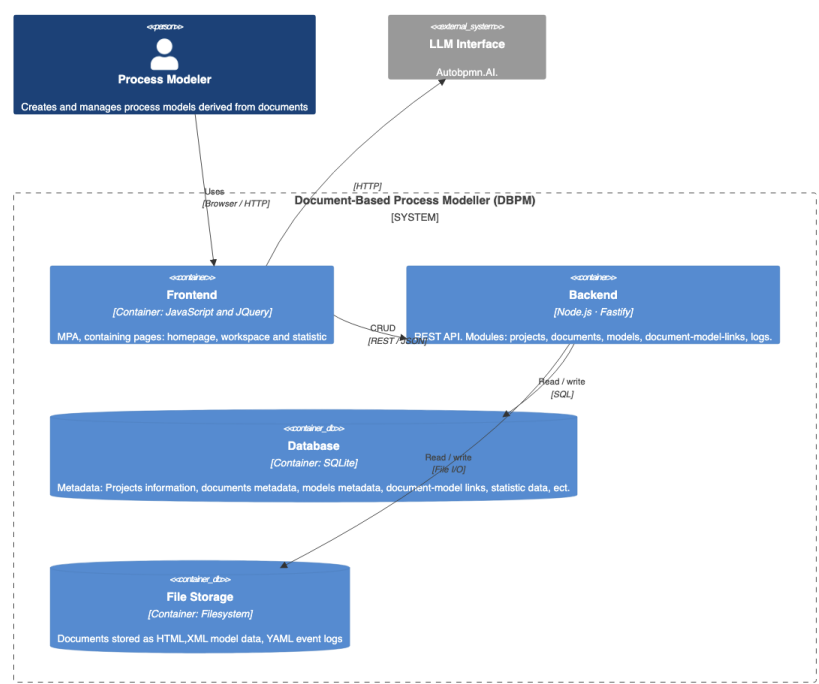
As the core part of the system, this section describes the modeling workflow that happens in the workspace page in Use Case Diagram. Figure: .

Design Difficulties

System Architecture

The system architecture is shown in the figure 5. The system follows the common web-application client-server pattern.

Figure 5  
System architecture



The system architecture of the document-based process modeling tool.

# Implementation

## Frontend

### Architecture

**Figure 6**

*Frontend codebase structure*



The frontend was implemented as a native web application using JavaScript, HTML, and CSS. This decision follows the design constraint defined in the previous chapter and keeps the client architecture lightweight while still allowing fine-grained control over document rendering and model-editor integration. In implementation terms, the codebase is organized into several layers. The HTML entry pages, such as `index.html` and `workspace.html`, provide page-specific containers. Each page then loads an initialization module that validates URL state, imports the required UI modules, and starts the runtime flow. The main workspace entry point is responsible for validating the project identifier, loading the page-specific UI modules, and triggering the initial data load for the current project.

The structure shown in Figure 6 reflects the realized separation of concerns. The `api` folder encapsulates communication with the backend REST endpoints. The `pages` folder contains the page-level composition logic, especially the workspace page where the end-to-end modelling workflow takes place. Inside the workspace page, the implementation is divided into services, stores, UI modules, utilities, and CSS resources. The `modules` folder contains domain-specific logic for document handling and process-model rendering, while `shared` provides reusable utilities such as generic state management, DOM helpers, and small widgets. This organization proved sufficient without introducing a frontend framework, because the application logic is centered on a small number of coordinated views rather than on a very large component tree.

At runtime, the workspace page acts as the central frontend container. After the project is validated, the workspace service loads the metadata for documents, models, and subprocess relationships, initializes the corresponding stores, and selects an initial document if available. Subsequent user actions, such as opening a document version, generating a model, switching to a historical version, or navigating to a subprocess, are then handled through the same orchestration layer. As a result, the frontend behaves as a workspace-centered reactive system rather than as a collection of isolated pages.

The simplified runtime flow is shown in Figure 7. It illustrates how page initialization, service orchestration, and store updates are chained during startup.

**Figure 7**  
*Simplified workspace initialization flow*

```

projectId = readProjectIdFromUrl()
if not projectId:
    redirectToHome()

validateProject(projectId)
loadWorkspaceUiModules()

documentsMeta, modelsMeta, subprocessLinks = fetchProjectComponents(
    projectId)
documentsStore.init(documentsMeta)
modelsStore.init(modelsMeta)
projectGraphStore.init(documentsMeta, modelsMeta, subprocessLinks)

if documentsMeta is not empty:
    latestDocument = documentsMeta[-1]
    documentService.loadVersion(latestDocument.latestVersionId)
    workspaceStore.setViewedDocument(latestDocument)

```

### ***State Management***

To coordinate the document list, document viewer, model editor, and project graph without introducing strong coupling between them, the frontend implements a lightweight publish-subscribe state-management mechanism. The base implementation is intentionally small and is shown in Figure 8. Each store keeps its internal state and a set of subscriber callbacks. Whenever the state changes, a patch object is emitted to all subscribers. This design keeps updates explicit and avoids hidden side effects in the UI layer.

The store abstraction is specialized for the needs of the workspace. A generic entity store manages collections by identifier and supports operations such as add, update, delete, selection, and bulk-edit state. On top of it, a version-aware entity store adds support for version histories and latest-version resolution, which is required by both documents and process models. Concrete stores then hold the active workspace context, document metadata, rendered document content, model metadata,

model-editor state, and the project graph. In practice, this means that UI modules can subscribe only to the changes relevant to them, while services remain responsible for orchestrating state transitions across multiple stores.

This custom pub-sub approach was chosen because the frontend requires reactive synchronization across multiple panes, but does not require the full abstraction overhead of a large framework. It also maps well to the workspace workflow. For example, switching the displayed document updates the viewed-document state, clears obsolete viewer state, loads the content and links of the selected version, and can optionally reset the model editor if the context has changed. The patch-based notification model makes these transitions observable and keeps the data flow understandable during development and debugging.

**Figure 8**  
*store implementation*

```
export class Store {
  constructor(initialState = {}) {
    this.state = { ...initialState };
    this.subscribers = new Set();
  }

  subscribe(fn) {
    this.subscribers.add(fn);
    return () => this.subscribers.delete(fn);
  }

  notify(patch) {
    this.subscribers.forEach((fn) => fn(patch));
  }
}
```

### ***Document Processing***

Document upload is handled partly on the frontend so that heterogeneous source formats can be normalized before they enter the main interaction flow. The implementation accepts PDF, DOC,

DOCX, and plain-text input. For Word documents, the content is converted to HTML and inserted into the viewer as structured markup. For PDF files, the preprocessing logic is more involved because the browser cannot directly provide a robust text structure for later selection and re-anchoring.

The PDF pipeline first extracts positioned text items from each page and normalizes their geometry, font size, and textual content. These low-level items are then grouped into lines according to their vertical alignment, after which several heuristics classify the resulting lines as headings, list items, or normal paragraph text. Adjacent lines are further merged into blocks based on indentation, vertical spacing, and font changes. Finally, the processed blocks are rendered as semantic HTML sections. This transformation is necessary because later frontend operations depend on stable DOM text ranges. Selection serialization, overlay rendering, and re-anchoring all become significantly more reliable when the input is available as normalized HTML instead of raw file data.

This preprocessing step is also an implementation trade-off. Some document layout information is simplified during conversion, especially in PDF files with complex formatting. However, for the purpose of document-based process modelling, preserving a readable and structurally coherent text flow is more important than preserving exact visual fidelity. The generated HTML therefore acts as an interaction-oriented representation of the document rather than as a pixel-accurate viewer.

### ***Text Selection Rendering***

One of the central frontend mechanisms is the transformation of browser text selections into persistent, version-aware linkable objects. When the user selects a passage in the document viewer, the range is serialized into a text-position selector together with a text-quote selector. This combination follows the idea of robust text anchoring from the W3C Web Annotation Model [19]. The text-position selector provides an exact range within the current text stream, while the text quote preserves the selected wording and can later support validation or re-anchoring after document updates.

Figure 9 shows the core implementation used to convert a browser range into a persistent selector object. This code is small, but it is central because it defines the representation that later connects document passages, stored links, and model generation requests.



**Figure 9***Core implementation of selection serialization*

```

export function serializeRange(range) {
  const root = getRoot();
  const textPosition = textPos.fromRange(root, range);
  const textQuoteSelector = textQuote.fromTextPosition(root, textPosition
    );

  return {
    textPosition,
    textQuote: textQuoteSelector,
  };
}

```

When a stored selection has to be displayed again, the frontend resolves the serialized selector back into a DOM range. This deserialization step is essential for loading existing links, reviewing older model versions, and visualizing the relation between documents and process models. The serialized form therefore acts as the boundary between the persistent representation stored by the system and the transient range object used by the browser at runtime.

**Virtual Overlay Strategy.** The visual rendering of selections is implemented as a virtual overlay rather than by directly wrapping the underlying text nodes. The selection module first collects client rectangles for the text range, filters out non-renderable rectangles, de-duplicates geometrically equivalent rectangles, and removes container rectangles that would otherwise lead to oversized highlight areas. An additional refinement is applied when a browser returns a single suspiciously tall rectangle for a multi-line selection. In this case, the implementation tokenizes the text and reconstructs more precise line-level rectangles. The resulting geometry is then used to draw separate overlay elements above the document content.

This strategy was chosen for two reasons. First, directly mutating the source DOM would complicate later re-rendering, selection updates, and version switches. Second, the document content is imported from heterogeneous sources, so keeping the original HTML structure untouched reduces

rendering side effects. The overlay-based approach therefore decouples interaction feedback from the content representation while still allowing precise highlighting.

**Link Update Detection.** The frontend also distinguishes between semantic changes in a linked selection and purely visual updates. When a link draft is compared against the current rendered selections, the implementation separately evaluates the text-position and text-quote information on the one hand and presentation properties such as highlight color or review status on the other hand. This leads to three explicit states: no change, color-only change, and text-changed. The distinction is important for document-version updates because a purely visual modification should not invalidate the semantic relation between a document passage and a model, whereas a changed text anchor may require review or re-anchoring.

### *Modeling*

The modeling flow is implemented as a coordinated interaction between the document viewer, the model service, and the model editor state. Once the user has selected one or more passages, the selected text is aggregated and optionally combined with an additional prompt. This generation input is then passed to the model service, which manages the asynchronous request to the LLM-backed generation endpoint, updates the editor status messages, and stores generation progress in the editor state.

The result of this process is model data represented as XML in the format required by the integrated CPEE-based editor. The frontend loads this data into the model editor store and renders it in the editing pane. At the same time, the service layer keeps the relation to the originating document version available so that the currently edited model can be synchronized with the document context. This is especially important when the user switches between new models, existing models, regenerated models, and historical versions.

Model versioning is therefore handled directly in the frontend interaction logic. A model may be newly generated from a fresh selection, regenerated from the linked document passage, or refined using an additional prompt. In all cases, the service layer maintains the distinction between the latest editable version and historical versions used for review. This version-aware coordination is one of the reasons why the modeling logic is implemented in a dedicated service rather than directly inside the UI modules.

### ***Rendering Customization***

Instead of implementing a process-model editor from scratch, the frontend reuses the CPEE layout and rendering infrastructure and extends it where necessary. The most important adaptation is the customization of the workflow adaptor base used by the renderer. The customized layer intercepts interactions with call nodes and subprocess nodes and dispatches additional browser events such as call-clicked, subprocess-hovered, subprocess-unhovered, and subprocess-double-clicked.

These custom events provide the bridge between the generic renderer and the specific needs of the document-based modeling workspace. In the implemented system, subprocesses are not only visual model elements but also navigable links to other models within the same project. The frontend therefore needs richer interaction signals than those originally provided by the renderer alone. By adding this customization layer, the system can support workspace-specific behavior, such as subprocess navigation and context-sensitive highlighting, while still preserving the original rendering engine and its XML-based editing capabilities.

Taken together, the frontend implementation is centered on keeping multiple representations synchronized: imported document content, persistent anchored selections, model versions, and interactive process visualizations. The layered organization, lightweight pub-sub stores, document normalization pipeline, overlay-based selection rendering, and renderer customization jointly enable the end-to-end workspace workflow described in the solution design.

## **Backend**

### ***Architecture***

The backend was implemented with Node.js and Fastify. This choice matches the lightweight architecture of the frontend while providing a practical HTTP server, straightforward route registration, and built-in support for request and response schema validation. At application startup, the server creates the required persistence directories, registers CORS and static-file handling, and mounts the domain-specific route modules for projects, documents, models, and document-model links. In this sense, the backend acts both as an API server and as the delivery layer for the frontend and persisted document/model assets.

The realized backend structure follows a layered route-service-repository pattern. Route modules define the REST endpoints and bind them to JSON schemas. Service modules contain the business logic, such as creating new document versions, copying or updating document-model links, generating model-version metadata, recording generation attempts, and handling soft-delete or restore operations. Repository modules encapsulate persistence details and isolate SQL access from the service layer. This separation keeps the HTTP layer small and allows the more involved versioning and synchronization logic to be implemented in a reusable way.

The schema layer is particularly important because the system processes structured selection data exchanged with the frontend. As shown in Figure 10, selectors are validated explicitly before entering the service logic. This reduces the risk of malformed anchoring data and keeps the contract between frontend and backend stable. The same pattern is applied across modules for documents, models, and links, where both request bodies and route parameters are validated before processing.

**Figure 10**

*Example JSON schema used in the backend for text quotes*

```
const textQuoteSchema = {
  type: "object",
  required: ["exact"],
  additionalProperties: false,
  properties: {
    exact: { type: "string" },
    prefix: { type: "string" },
    suffix: { type: "string" },
  },
};
```

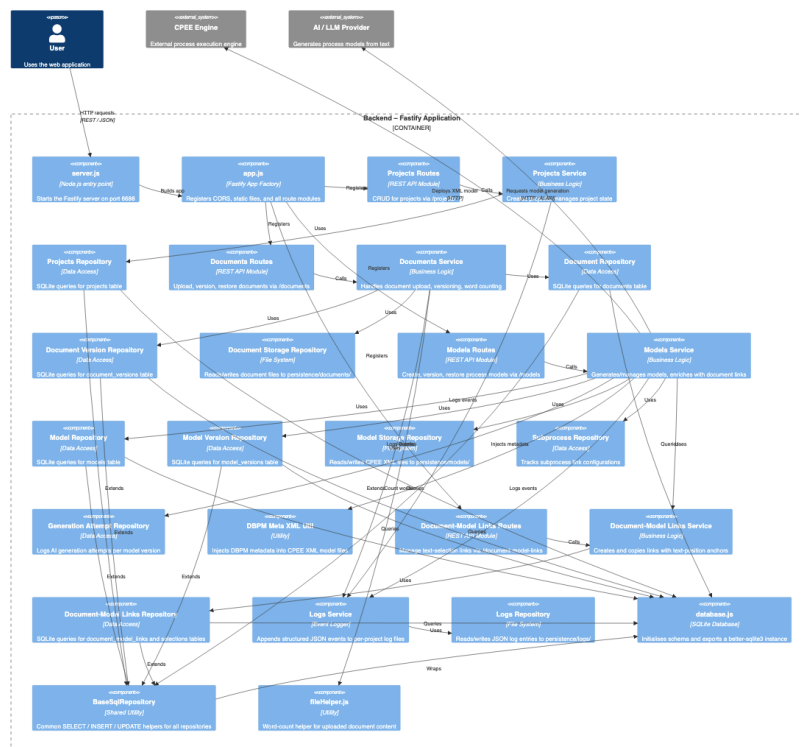
The backend implementation combines relational persistence with file-based storage. Metadata such as projects, documents, models, versions, links, subprocess references, generation attempts, and version events are stored in SQLite via the `better-sqlite3` library. By contrast, large document contents and model XML payloads are stored as files in the persistence directory. This hybrid design is deliberate. Relational storage supports queries over versions, links, and project-level statistics, whereas file storage avoids placing large HTML and XML payloads directly into

the relational schema. A shared base repository class provides reusable create, read, and update operations, including camelCase to snake\_case mapping and JSON serialization for structured columns.

The service layer implements the application-specific behavior that the frontend depends on. For example, when a new document version is created, the backend stores the new content, updates the latest-version pointer, copies the latest link information to the new document version, and schedules the corresponding model-link synchronization work. Likewise, when a new model or model version is created, the backend persists the XML payload, updates latest-version references, injects DBPM-specific metadata into the model data, stores the related document-model link, and records monitoring information such as generation attempts or manual update events. These operations involve several repositories at once, which is why they are implemented at the service level rather than in individual route handlers.

The backend also implements several cross-cutting concerns required by the overall system design. Soft deletion is used for projects, documents, subprocess links, and models so that historical information can remain available for restoration and analysis. Logging is handled as a dedicated module and is invoked from the service layer when relevant lifecycle actions occur. The project service additionally exposes aggregated component endpoints that bundle document metadata, model metadata, and subprocess links into a single response tailored to the workspace frontend. The resulting architecture is shown in Figure 11. Rather than exposing the database directly to the client, the backend serves as the consistency layer that preserves version relations, link integrity, and monitoring data across the full modelling workflow.

**Figure 11**  
Backend Component



The backend component of the document-based process modeling tool.

## Database Schema

Firstly, Project, Document, Model and Document Model Link are the minimal tables needed to support the proposed system. It is also how the system was implemented when it first adopted the database. The Document and Model tables were used to store metadata like name, words count of a document. Theoretically, the Document Model Link table can be stored as an attribute within the model version, since the system is currently designed to allow one model to be derived from only one document. But it is a decision made very early to store it in a separate table to easily support a potentially many-to-many (M2M) relationship in the future.

The Model Version table is added when the model versioning feature is about to be implemented. The Document Version table is added at the same time to prepare for supporting the Document Update feature, as both tables follow a similar logic at the DB operation level.

The Subprocess Link table is an additional table used to support the Project Graph. As this relationship information has been embedded in Model Data, the model visualization itself does not need this data. However, for the project graph, the relationship is hard to be parsed from the model data; therefore, such a table is designed to easily retrieve the process-subprocess relationship.

The Selection and Selection History tables are then added when designing the Document Update feature. Before the selection data (e.g., color, position) for a Document Model Link is stored as an attribute in the format of a stringified JSON object. To better support auto-reanchoring for document upload, Selection and Selection History are designed as separate tables. The separate Selection table can make selection data easier to retrieve for reanchor operations. Besides, a Selection History is designed to make it easier to track the quality of reanchor.

Finally, the Generation Attempt and Version Event tables are the two tables designed to monitor the system behavior. The Generation Attempt table is used to monitor the accept/decline decision for the LLM-generated output. The Version Event is mainly to store the manual edit behavior or a system auto-update behavior, e.g., selection reanchor for document update.

## Evaluation

### Evaluation Procedure

The developed system was evaluated with four modeling experts to assess its usability and perceived usefulness, as well as to gather feedback on potential improvements and additional features. The evaluation process included a short demonstration, task-based activities, and a questionnaire. Each evaluation session lasted approximately 40 minutes.

The session was structured as follows: It began with a demonstration to introduce the tool's end-to-end workflow and key features. After that, the participants were asked to complete two tasks. The first was an exploratory task based on a fictional B2B order-processing scenario, consisting of two wiki-style documents in DOCX format. It was designed to help participants familiarize themselves with the system. The second was a more realistic process modeling task based on an open-source SOP document<sup>2</sup> in PDF format. For both tasks, participants were not given strict instructions on

---

<sup>2</sup>DNRC SOP Document

how to proceed, allowing them to explore the system freely. Due to the known issue of the PDF file rendering, the link to the online version was provided. If time permitted, participants could additionally use the system with their own documents.

Following task completion, participants completed a questionnaire including the following open-ended questions:

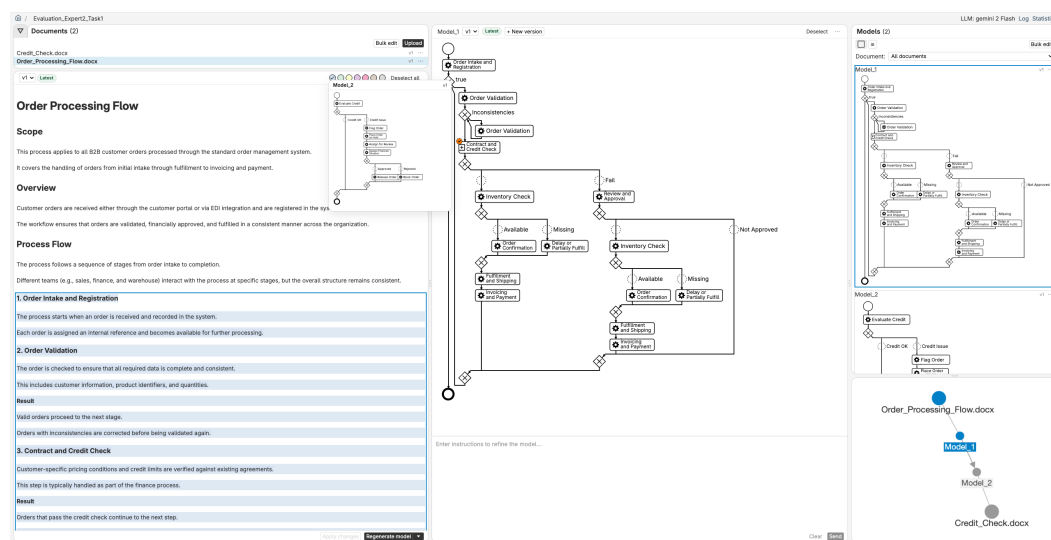
- (1) What aspects of the tool did you like while using it?
- (2) What aspects of the tool did you not like or find problematic?
- (3) What improvements or additional features would you most like to see in the tool?
- (4) If you needed to model processes from documents in the future, would you consider using this tool? Why or why not?

## Evaluation Results

### *Overall Perception and Task Completion*

Overall, participants perceived the tool as usable and intuitive. Figure 12 and Figure 13 show the modeling results from one expert for the first and second tasks, respectively.

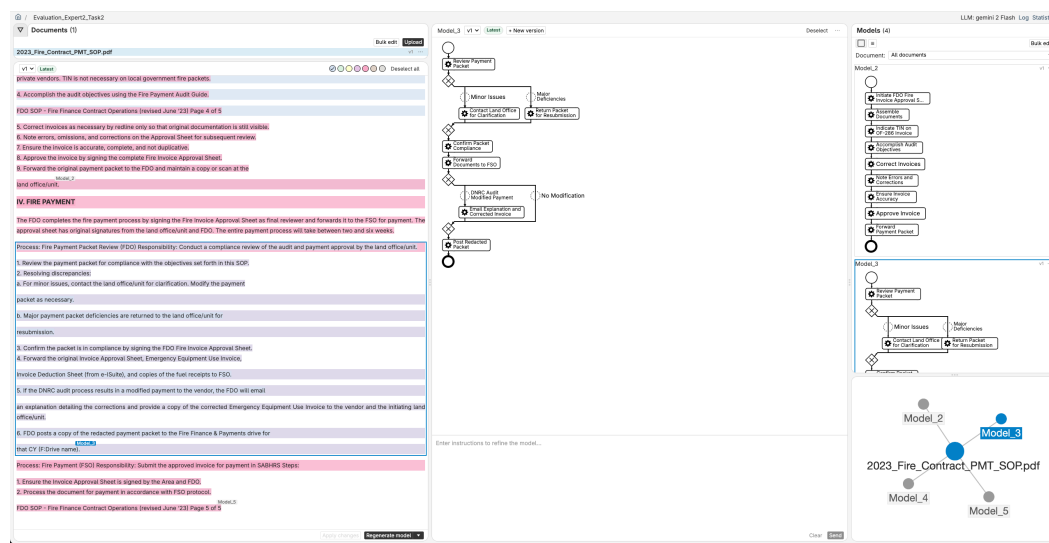
**Figure 12**  
*Modeling result*



Modeling result from an expert for the first task.



**Figure 13**  
*Modeling result*



Modeling result from an expert for the second task.

One interesting observation is that while most participants tended to start with, or at least try once, selecting larger portions or all of the text when reading a new document, one participant began by creating models from smaller segments. These different strategies are illustrated in Figure 14.

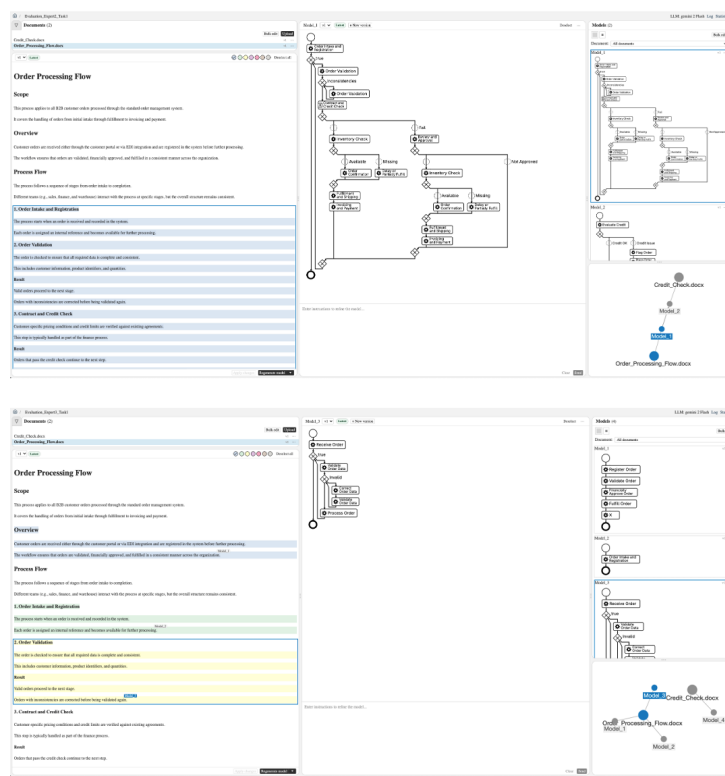
For detailed task completion and feedback, the participants showed varied interests and offered differing perspectives on the system. The result on each feature is summarized in Table 3, and will be described in detail in the following sections.

### **Positive Aspects**

Firstly, the following aspects are generally perceived positively by the participants: the performance of the integrated LLM service, the easy-to-establish process-subprocess relationship, and model version control.

Furthermore, several additional positive aspects were mentioned. First, the embedding of relevant information, such as selected text and prompt history, within the model's XML data is considered a well-integrated design feature. Secondly, logging and statistical information elicited interest among some participants due to its potential for further analysis. Thirdly, the project graph visualization was mentioned as helpful, especially for larger systems.

**Figure 14**  
*Different modeling strategies*



Different modeling strategies observed during task execution (above: larger part selection; below: step-by-step selection)

### ***Problems in Interaction and Use***

The following outlines the main problems and challenges reported by participants, as well as notices observed during the evaluation.

First, for the document's rendering and interaction, only one feedback regarding the readability issue of PDF documents was mentioned in feedback. But it is observed that one participant started with reading the original PDF online firstly and one participant tried to use browser-extension to search for keywords.

Regarding the modeling process design, two problems were mentioned. First, the step to start a new modeling by a explicit deselection step of the current model was mentioned. Two reply received in the feedback, but it is a feature generally perceived as not intuitive immediately and needs additional clarification. Second, due to the sufficient good quality of the initial model, the review step was

**Table 3**  
*Summary of Evaluation Results by Functionality*

| Functionality                      | Positive Aspects                |   | Problems and Limitations                                             |   |
|------------------------------------|---------------------------------|---|----------------------------------------------------------------------|---|
|                                    | Item                            | n | Item                                                                 | n |
| Document Rendering and Interaction |                                 |   | Readability issue                                                    | 1 |
|                                    |                                 |   | Expected AI-assisted search capability                               | 1 |
| Modeling Process                   |                                 |   | Unintuitive to start a new modeling by deselecting the current model | 2 |
|                                    |                                 |   | Unnecessary review step (of the initial generated model)             | 2 |
| Integrated LLM Service             | Fast                            | 2 | Lack of transparency in the conversational modeling logic            | 2 |
|                                    | Good quality                    | 2 |                                                                      |   |
| Model Versioning                   | Helpful                         | 3 | Expected version comparison                                          | 1 |
| Augmented Model Data               | Well-considered                 | 2 |                                                                      |   |
| Text-Model Link                    | Clear                           | 1 | Expected fine-grained traceability                                   | 2 |
| Process-Subprocess Relationship    | Convenient                      | 3 |                                                                      |   |
| Project Graph                      | Helpful for larger systems      | 1 | Unclear node types and version representations                       | 2 |
|                                    |                                 |   | Expected more expressive visualization                               | 1 |
| Log and Statistics                 | Potentially useful for analysis | 1 |                                                                      |   |

n = number of participants who mentioned each item

mentioned by one participant as totally unnecessary as there is version control and by another one as the initial generated model was mentioned as an unnecessary step.

Another main problem mentioned and observed is that the system's conventional modeling logic is not transparent to users and needs additional explanation. While the initial model generation from text was relatively clear, the model modification process was less transparent. They did not understand how their inputs were processed or how the system used the selected text and existing model.

### ***Potential Improvements and Additional Features***

Besides the problems found during system use, many valuable insights were collected.

For the existing features, improvements to the project graph visualization were mentioned most often. This includes clearer differentiation and identification of elements (i.e., what type of element a node is and which version it represents), as well as using more expressive visual designs, such as size, color saturation, and distance, to show additional information beyond type.

For document-related features, an AI-assisted document search and interaction function was suggested. This would allow users to query the document by providing an instruction, after which the retrieved parts can be automatically selected and used directly or as a basis for further adjustment for model generation.

For the process modeling features, one notable desired improvement emphasized by the participants is the need for finer-grained traceability between source text and model. When selecting larger text passages, only a subpart of the text is actually used and reflected in the generated model, and it is unclear which parts are actually used. Therefore, a clearer indication of the actual text used for model generation is required, ideally at the level of individual tasks. Secondly, model comparison was suggested as a valuable feature to help users understand and verify changes between different model versions, making it easier to identify modifications and track model iterations over time. Beyond this, participants also expected more general advanced AI modeling capabilities, including explainability of the generated model through conversational interaction with chatbots, as well as support for model quality evaluation and automatic improvement.

### ***Future Use Intention***

Three participants indicated an intention to use the system in scenarios involving the creation of multiple models. In such cases, the system's features, such as text-model link, process-subprocess relationship, and model management functionalities, were considered helpful. One participant additionally raised the concern that automatically generated models may bypass the user's need to fully understand the underlying document, potentially introducing bias. Hence, the fine-grained traceability, as suggested by the participant, could help mitigate this problem. Another participant again pointed out the need for the AI-assisted document search capability as suggested.

## Discussion

This chapter discusses the evaluation’s findings and reflects on their implications for system design.

### Usability and User Experience

Among these usability issues, some can be mitigated and improved through design, while others are more challenging to be improved for the system.

The first finding is that document readability is key to the tool’s design, as it aims to enable users to start using it by reading. If a user doesn’t have a good reading experience, turns to reading the document in their familiar way, and then finds the same text position within the tool, it will significantly affect the tool’s designed usage scenario and reduce efficiency.

For the interaction design, some of the improvement ideas have been made up; e.g., to avoid the unnecessary review step, we can easily offer a project settings feature. And for the transparent AI input or system logic, it can be improved by showing instructions, e.g., using a switch button to give the user more flexible control over the AI input. In contrast, as this tool introduces a new way of process modelling involving multiple documents and multiple models, e.g., deselecting the deselect current model to start a new model step, is quite representative, without coming up with a good solution, therefore, it requires users to be familiar with the new interaction and workflow, which raises the challenge for the system.

### User Expectations

During the evaluation, participants raised more advanced expectations, which are out of the current system’s scope, but are worth further discussion.

Firstly, the expected fine-grained traceability between text and model represents an advanced requirement that goes beyond the capabilities of current LLM-based process modelling tools. However, it is a particularly important feature in the context of document-based process modelling, as the system is designed to facilitate a easier verification for the traceability of the AI-generated model. Therefore, exploring approaches to enable fine-grained traceability can be considered a valuable direction for future work.

Secondly, the expectation of integrated AI-powered document intelligence is another advanced requirement that has not yet been included in the current system. However, it represents a promising and valuable direction for future development, as such capabilities could further enhance the system's efficiency and perceived usefulness. In particular, enabling intelligent document understanding, contextual retrieval, and semantic linking between textual content and modelling elements could significantly streamline the modelling workflow and improve the overall user experience.

### **Limitations**

Due to the small sample size ( $n=4$ ), all of whom are modelling experts, the study did not include the perspective of domain experts. Besides, although the material provided is a real-world SOP document, it is a really well-written one with appropriate structure and not too long, 5 pages. So the document readability issue could affect the tool's usability more.

Besides, the evaluation didn't mention it as a limitation that the system is currently designed so that a model can be generated from a single document, without enabling cross-document modeling, which may require a more complex user interaction design.

## **Conclusion**

This work is motivated by the inconvenience of state-of-the-art AI-assisted process modeling tools, which typically only provide a text input box and lack of support for document-based sources. To resolve this issue, we designed, implemented, and evaluated an integrated web-based process modeling tool, especially for the document-based process modeling scenario, offering a seamless, efficient, and user-friendly workflow.

We first revisit the research questions:

RQ1 How do existing tools support AI-assisted process modelling with documents?

A thorough systematic review of existing tools and approaches for AI-assisted process modelling is conducted. The review shows that most of the existing tools still focus on model generation capabilities and quality. There is a trend to augment the input model with audio. However, the use case of creating process models from documents or a segment of documents remains missing.

RQ2: What functionalities should a system provide to support AI-assisted process modelling from documents, and how should these functionalities be designed and integrated in a seamless and user-friendly manner?

We derived minimal core requirements as follows: a unified workspace for documents and models; direct text selection from documents as LLM input; side-by-side visualization with explicit text–model traceability; and subprocess relationship management. Additional enhanced features include LLM-based model revision, model versioning, document update support, bulk management, and interactive project graph visualization. The design process showed that extending functionality increases integration complexity and may negatively affect usability, requiring careful trade-offs.

RQ3 How do users perceive the benefits and limitations of the system?

An evaluation with four process modeling experts indicates that the system is generally intuitive and usable, with particular value for creating and exploring multiple models from documents, supported by features such as text–model links and process–subprocess links. The evaluation implies two valuable future work directions: fine-grained text–model traceability and AI-powered document intelligence, both of which have particular potential to improve efficiency and user experience.

Overall, this work demonstrates the feasibility and practical value of supporting document-based process modeling through the developed prototype. It introduces a novel input modality that enables direct text selection from uploaded documents as LLM input. Future work will focus on further improving the system’s usability, extending support to additional document types, and conducting research in fine-grained traceability and the integration of AI-powered document intelligence.

## Bibliography

- [1] N. Klievtsova, J.-V. Benzin, J. Mangler, T. Kampik, and S. Rinderle-Ma, “Process modeler vs. chatbot: Is generative ai taking over process modeling?” In *International Conference on Process Mining*, Springer, 2024, pp. 637–649.
- [2] A. R. Hevner, “A three cycle view of design science research,” *Scandinavian Journal of Information Systems*, vol. 19, no. 2, pp. 87–92, 2007.
- [3] H. Kourani, A. Berti, D. Schuster, and W. M. Van Der Aalst, “Promoai: Process modeling with generative ai,” *arXiv preprint arXiv:2403.04327*, 2024.
- [4] J. Köpke and A. Safan, “Introducing the bpmn-chatbot for efficient llm-based process modeling,” in *BPM (Demos/Resources Forum)*, 2024, pp. 86–90.
- [5] J. T. Licardo, N. Tanković, and D. Etinger, “Bpmn assistant: An llm-based approach to business process modeling,” *Applied Sciences*, vol. 16, no. 5, p. 2213, 2026.
- [6] J. Köpke and A. Safan, “Efficient llm-based conversational process modeling,” in *International conference on business process management*, Springer, 2024, pp. 259–270.
- [7] C. Lauer, P. Pfeiffer, and N. Mehdiyev, “Human-centered evaluation of an llm-based process modeling copilot: A mixed-methods study with domain experts,” *arXiv preprint arXiv:2603.12895*, 2026.
- [8] S. Pardo Gutierrez, L. Alrabie, G. Di Fede, M. Vitali, and S. Andolina, “A direct manipulation interface for llm-based process modeling,” in *Proceedings of the 16th Biannual Conference of the Italian SIGCHI Chapter*, 2025, pp. 1–3.
- [9] L. F. Hörner, “Towards an llm-based conversational framework for business process modeling: Research approach and preliminary results,” in *International Conference on Advanced Information Systems Engineering*, Springer, 2025, pp. 286–293.
- [10] L. F. Hörner, M. Möller, and M. Reichert, “Automatically generating bpmn 2.0 process models from natural language process descriptions: Challenges, framework, quality assessment,” *Business & Information Systems Engineering*, pp. 1–25, 2026.
- [11] C. Ziche and G. Apruzzese, “Llm4pm: A case study on using large language models for process modeling in enterprise organizations,” in *International conference on business process management*, Springer, 2024, pp. 472–483.



- [12] N. Klietsova, M. Ehrendorfer, J. Mangler, and S. Rinderle-Ma, “Autobpmn. ai: Conversational process modeling and automation,” *CEUR Workshop Proceedings*, vol. 4032, pp. 304–311, 2025, issn: 1613-0073.
- [13] A. Safan and J. Köpke, “Bpmn-chatbot++: Llm-based modeling of collaboration diagrams with data,” in *Proceedings of the BPM*, 2025.
- [14] A. Costa, A. Wimmer, and L. Pufahl, “Llm4bpmngen: A tool for bpmn generation with llms,” 2025.
- [15] A. Nour Eldin, N. Assy, O. Anesini, B. Dalmas, and W. Gaaloul, “Nala2bpmn: Automating bpmn model generation with large language models,” in *International conference on cooperative information systems*, Springer, 2024, pp. 398–404.
- [16] A. Safan and J. Köpke, “A framework for llm-based conceptual modeling: Application to bpmn collaboration diagrams,” in *ER2025: companion proceedings of the 44th international conference on conceptual modeling: industrial track, ER Forum, 8th SCME, Poitiers*. <https://ceur-ws.org>, vol. 4099, 2025.
- [17] Wikipedia contributors, *Functional requirement*, [Online; accessed 14-April-2026], 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Functional\\_requirement](https://en.wikipedia.org/wiki/Functional_requirement).
- [18] D. Randall Brandt, “How service marketers can identify value-enhancing service elements,” *Journal of Services Marketing*, vol. 2, no. 3, pp. 35–41, 1988.
- [19] W3C, “Web annotation data model,” World Wide Web Consortium, Tech. Rep., 2017, Accessed: 2026-04-05. [Online]. Available: <https://www.w3.org/TR/annotation-model/>.

## Appendix

- *Upload Documents.* The user uploads multiple documents in textual format, e.g., TXT, DOCX, PDF.
- *Select Text Passages.* The user selects multiple passages from one uploaded document, then clicks the Generate button to .

It shall allow users to select multiple passages from one document to serve as input for LLM-based process model auto-generation.

It shall provide a clear visual link between the text and the generated model.

It shall embed the source text within the generated process model for traceability purposes.

It shall allow users to establish process-subprocess relationships between models.

It shall record user behavior and interactions with the system for analysis purposes.

One additional requirement is that the system shall be able to support document update.

### *Design Constraints*

- It shall be implemented as a web-based application.
- The frontend shall be developed using native web technologies (JavaScript, HTML, and CSS).
- *Process Model Visualization.* The visualization of the process model shall be implemented using the CPEE Layout<sup>3</sup> library.
- *LLM Service Integrati.* The LLM-backed process model generation shall be implemented using the API from AutoBPMN.AI.

---

<sup>3</sup><https://github.com/etm/cpee-layout>